

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ТЕОРИЯ

ИГРОВЫЕ СОСТОЯНИЯ (GAME STATES)

Любая игра имеет несколько состояний: меню, игра, пауза, победа, поражение. В программировании мы называем это 'состояниями игры' или game states.

Каждое состояние меняет поведение игры и то, что видит игрок на экране. Например, в состоянии 'меню' показывается кнопка 'Начать', а в состоянии 'игра' - игровое поле.

Мы управляем состояниями с помощью переменных-флагов (Boolean) или специальных перечислений (enum class). Флаг isDiving = true/false показывает, погружается ли лодка.

УПРАВЛЕНИЕ РЕСУРСАМИ

В стратегических играх ресурсы - это то, что нужно собирать и расходовать. Золото тратится на улучшения, кислород уменьшается при погружении, здоровье теряется в бою.

Хорошая система ресурсов заставляет игрока принимать решения: 'Стоит ли рисковать и погрузиться глубже ради большего золота?' или 'Лучше вернуться сейчас и купить улучшение?'

В Kotlin мы используем coerceAtLeast(0f) чтобы ресурс никогда не стал отрицательным, и coerceAtMost(max) чтобы он не превысил максимум.

УСЛОВНЫЕ ВЫРАЖЕНИЯ: WHEN И IF

when в Kotlin - это мощный аналог if/else. Он проверяет несколько условий и выполняет соответствующий блок кода.

when можно использовать как выражение (возвращает значение) или как оператор (просто выполняет код). В этом уроке мы используем when для определения, какое улучшение покупать по ID.

if(condition) A else B - это сокращенная запись условия. Мы используем её для определения цвета индикаторов: красный если мало кислорода, зеленый если много.

МОДУЛЬНЫЙ ИНТЕРФЕЙС

Когда экран содержит много одинаковых элементов (например, строки магазина), лучше вынести их в отдельный компонент.

Компонент SubUpgradeRow принимает данные конкретного улучшения и рисует его. Это избавляет нас от дублирования кода.

LazyColumn - это умный список, который создает только те элементы, которые видны на экране. Если у тебя 100 улучшений, LazyColumn создаст только 5-6 видимых.

ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 1: 1. Описание модели улучшений (вверху файла)

```
data class SubUpgrade(  
    val id: String,  
    val name: String,  
    val level: Int,  
    val baseCost: Long,  
    val icon: String  
) {  
    val cost: Long  
        get() = baseCost + (level * (baseCost * 0.6)).toLong()  
}
```

ЭТАП 2: 2. Компонент UI: Карточка Улучшения (SubUpgradeRow)

```
@Composable  
fun SubUpgradeRow(upgrade: SubUpgrade, canAfford: Boolean, onUpgradeClick: () ->  
Unit) {  
    Card(  
        modifier = Modifier.fillMaxWidth(),  
        colors = CardDefaults.cardColors(containerColor = Color(0xFF0F1C3F))  
    ) {  
        Row(  
            modifier = Modifier.fillMaxWidth().padding(10.dp),  
            horizontalArrangement = Arrangement.SpaceBetween,  
            verticalAlignment = Alignment.CenterVertically  
        ) {  
            Column {  
                Text(upgrade.icon + " " + upgrade.name, color = Color.White, fontWeight =  
                    FontWeight.Bold)  
                Text("Уровень: " + upgrade.level, color = Color(0xFF4FC3F7), fontSize = 12.sp)  
            }  
            Button(onClick = onUpgradeClick, enabled = canAfford,  
                colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF00E676))) {  
                Text(upgrade.cost.toString() + " ⚙", fontSize = 12.sp, color = Color.Black)  
            }  
        }  
    }  
}
```


ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 3: 3. Объявление главного экрана

```
@Composable
fun SubmarineExplorerScreen() {
    // Весь код до превью - ВНУТРИ этой функции
```

ЭТАП 4: 4. Состояние игры (Переменные)

```
var gold by remember { mutableStateOf(100L) }
var depth by remember { mutableStateOf(0f) }
var oxygen by remember { mutableStateOf(100f) }
var isDiving by remember { mutableStateOf(false) }

var hullLevel by remember { mutableStateOf(1) }
var tankLevel by remember { mutableStateOf(1) }
var engineLevel by remember { mutableStateOf(1) }
```

ЭТАП 5: 5. Математические расчеты (Динамические параметры)

```
val maxOxygen = tankLevel * 100f
val maxDepth = hullLevel * 200f
val diveSpeed = engineLevel * 6f
val oxygenUsageRate = 3f / tankLevel
```


ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 6: 6. Игровой цикл погружения (LaunchedEffect)

```
LaunchedEffect(isDiving) {
    if (isDiving) {
        while (isDiving) {
            delay(500)
            oxygen = (oxygen - oxygenUsageRate).coerceAtLeast(0f)

            if (oxygen <= 0f) {
                isDiving = false
                depth = 0f
                gold = (gold - 50L).coerceAtLeast(0L)
            } else if (depth < maxDepth) {
                depth = (depth + diveSpeed).coerceAtMost(maxDepth)
                gold += (depth / 30).toLong()
            }
        }
    }
}
```

ЭТАП 7: 7. Главный макет экрана (Box и Column)

```
Box(
    modifier = Modifier.fillMaxSize().background(Color(0xFF050E1E)),
    contentAlignment = Alignment.TopCenter
) {
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        /* Сюда добавляем элементы в следующих шагах */
    }
}
```


ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 8: 8. Информационная панель (Метрики)

```
Text("ПОДВОДНАЯ МИССИЯ  БЕЗДНА ", color = Color.White, fontSize = 18.sp,
fontWeight = FontWeight.Bold)

Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.SpaceBetween) {
Text("Глубина: ${depth.toInt()} / ${maxDepth.toInt()} м", color =
Color(0xFF4FC3F7))
    Text("Золото: $gold 🪙", color = Color(0xFFFFD54F))
}

    Column(modifier = Modifier.fillMaxWidth()) {
Text("Кислород: ${oxygen.toInt()}/${maxOxygen.toInt()}", color = Color.LightGray,
fontSize = 12.sp)
    Spacer(modifier = Modifier.height(4.dp))
    LinearProgressIndicator(
        progress = { oxygen / maxOxygen },
        modifier = Modifier.fillMaxWidth().height(10.dp),
        color = if (oxygen < 30f) Color.Red else Color(0xFF00E676)
    )
}
```

ЭТАП 9: 9. Панель управления (Кнопки)

```
Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.spacedBy(12.dp)) {
Button(onClick = { isDiving = !isDiving }, modifier = Modifier.weight(1f),
        colors = ButtonDefaults.buttonColors(
containerColor = if (isDiving) Color(0xFFD32F2F) else Color(0xFF028D11)) {
    Text(if (isDiving) "ВСПЛЫТИЕ" else "ПОГРУЖЕНИЕ")
        }
)

Button(onClick = { oxygen = maxOxygen; depth = 0f }, modifier =
Modifier.weight(1f),
        enabled = !isDiving,
        colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF455A64))) {
    Text("ЗАПРАВКА")
        }
}
```


ПОДВОДНАЯ МИССИЯ: БЕЗДНА

МОДУЛЬ: ИГРОВЫЕ СОСТОЯНИЯ, РАСХОД РЕСУРСОВ И МОДУЛЬНЫЙ ИНТЕРФЕЙС | ID: 6

ЭТАП 10: 10. Список улучшений (LazyColumn)

```
val upgrades = listOf(
    SubUpgrade("hull", "Прочность корпуса", hullLevel, 40L, "❤️ "),
    SubUpgrade("tank", "Кислородные баллоны", tankLevel, 50L, " "),
    SubUpgrade("engine", "Турбо-двигатель", engineLevel, 70L, " ")
)

LazyColumn(modifier = Modifier.fillMaxWidth().weight(1f), verticalArrangement =
Arrangement.spacedBy(8.dp)) {
    items(upgrades) { item ->
        SubUpgradeRow(
            upgrade = item,
            canAfford = gold >= item.cost && !isDiving,
            onUpgradeClick = {
                gold -= item.cost
                when (item.id) {
                    "hull" -> hullLevel++
                    "tank" -> { tankLevel++; oxygen = tankLevel * 100f }
                    "engine" -> engineLevel++
                }
            }
        )
    }
}
```

ЭТАП 11: 11. Предпросмотр экрана (Preview)

```
// Пишется СНАРУЖИ функции SubmarineExplorerScreen!
@Preview
@Composable
fun SubmarineExplorerPreview() {
    MaterialTheme {
        SubmarineExplorerScreen()
    }
}
```

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Добавь Систему Сонара: создай переменную sonarActive. При включенном сонаре (расходует 1.5 дополнительного кислорода за тик) с шансом 20% игрок находит дополнительное золото (+100 монет).